

ACCOUNTING

A disciplined financial engine.

The ledger, the sign convention, share-capital accounting, profit accrual and payout — and the transaction and idempotency rules that keep every rupee accounted for.

DOCUMENT

Accounting & Finance

AUDIENCE

Finance · Engineering

VERSION

1.0

PRECISION ENGINEERING FOR MODERN FINANCE

THE LEDGER

| One audit trail for every rupee that moves

FinFlo keeps two complementary records. **FinancialTransaction** is the audit ledger — a row is written for every piece of auditable money movement (repayments, disbursements, fees, profit, transfers). **ActivityLog** records what users *did*, separate from what money *moved*. Money logic itself lives in `src/services/`, not in controllers, so the rules that govern balances are written once and reused everywhere.



FinancialTransaction

The auditable money ledger: type, category, amount, status, and references back to the loan, member, customer, or originating document.



ActivityLog

The record of user actions — who did what, when — kept distinct from the money trail.

Ledger amounts are rounded to whole rupees at rest, and reversals are first-class: a reversed entry links to its reversal transaction with a reason, so corrections are themselves auditable.

TRANSACTION DIRECTION

| A single source of truth for credit vs debit

Whether a transaction is a **credit** (an inflow that raises a balance, shown with `+`) or a **debit** (an outflow that lowers it, shown with `-`) is decided in one place — the canonical `transactionDirection` module and its `isCreditType` helper. Components and reports never re-declare their own credit/debit lists; a new transaction type is added once, here, rather than in every view. This prevents the classic drift where the same type renders `+` in one screen and `-` in another.

TRANSACTION TYPE	PERSPECTIVE	DIRECTION
<code>deposit</code>	Member wallet	Credit (+) raises balance
<code>loan_disbursement</code>	Member wallet	Credit (+) proceeds in
<code>profit</code>	Member wallet	Credit (+) profit in
<code>withdrawal</code>	Member wallet	Debit (-) lowers balance
<code>transfer_send</code> / <code>external_send</code>	Member wallet	Debit (-) money out
<code>transfer_receive</code> / <code>external_receive</code>	Member wallet	Credit (+) money in
<code>income</code> / <code>credit</code>	Business books	Credit (+) ledger inflow
<code>expense</code> (e.g. profit distribution)	Business books	Debit (-) ledger outflow

The direction is relative to perspective: a profit distribution raises a member's wallet but is an expense to the business books — the same helper answers both correctly.

SHARE CAPITAL ACCOUNTING

Member shares are a liability — treated so everywhere

A member's **share balance** is capital the member has placed with the business. Throughout FinFlo it is accounted for as a **liability** — money the institution owes back — never as the institution's own income. The share balance and profit are kept separate from the member's main wallet and are never auto-deducted to service a loan.

How share cash is reported

The cash attributable to member shares is reported as `shareBalance - totalShareProfit`. Profit credited to a member's shares is not the institution's cash, so it is subtracted out. There is deliberately **no separate** `totalShareWithdrawn` field — withdrawals are already reflected in the running `shareBalance`, so the figure must be derived exactly as above and not from an invented field.

SAVINGS PROFIT

| Accrued daily, distributed on schedule

Savings accounts earn profit through scheduled jobs. A daily **saving-profit accrual job** (`runSavingProfitAccrual`) accumulates each account's pending profit, and a monthly **distribution job** (`runMonthlySavingProfitDistribution`) moves accrued profit into the member's saving balance. Both run in the Asia/Karachi timezone, and the last-accrual timestamp de-duplicates the daily run so profit is never credited twice.

Per-account rate

Profit is computed from each account's configured profit rate, so different products and members can carry different rates.

Accrue then distribute

Profit accrues daily into a pending figure and is distributed on the monthly cadence — separating the two keeps daily computation cheap and distribution atomic.

De-duplicated

A stored last-profit timestamp guards the daily job, so a re-run on the same day cannot double-credit.

DISTRIBUTIONS & PAYOUTS

| Getting money out, safely

Profit and capital owed to members and investors are paid out through the `payoutService`, which initiates outgoing real-time transfers (over Raast or another provider) to a bank account or wallet. A profit distribution is recorded as such on the business books — an *expense* in ledger terms — and credited to the recipient from their own perspective.

payoutService

Handles the outbound transfer: signs the request, calls the provider, and returns a transaction reference for the disbursement.

Recorded both ways

Each distribution lands in the ledger as a business outflow and raises the member's or investor's balance — the single direction helper keeps both signs consistent.

INTEGRITY & IDEMPOTENCY

| The rules that keep the books correct

Transactions and idempotency are mandatory

Any multi-document money operation runs inside a MongoDB session via `startSession` / `withTransaction`. Operations are made idempotent with guarded `findOneAndUpdate` filters, atomic `$inc` updates, and `ProcessedWebhookEvent` deduplication so a retried or replayed event cannot double-apply. Critically, `abortTransaction()` is called on **every** early-return and error path — a real bug class this prevents, because a leaked open transaction would hold database locks. Either the whole money movement commits, or none of it does.

GUARD	WHAT IT PROTECTS AGAINST
Session / transaction	Partial writes — a balance updated without its ledger row, or vice versa.
Guarded <code>findOneAndUpdate</code> / <code>\$inc</code>	Lost updates and double-application under concurrency.
ProcessedWebhookEvent dedup	Replayed webhooks applying the same money movement twice.
<code>abortTransaction()</code> on every exit	Leaked open transactions holding locks after an early return.

BALANCES & ACCOUNTS

The balances a member carries

A member holds several distinct balances, each with its own purpose and its own account number. They are kept separate so that genuine capital, borrowed proceeds, savings, and shares are never conflated in a report.

ACCOUNT / BALANCE	NUMBER PREFIX	WHAT IT HOLDS
Current account (<code>currentBalance</code>)	CUR	The main wallet: invested capital and loan proceeds, less withdrawals, plus profit. Loan proceeds are tracked separately so reports don't count borrowed money as deposits.
Saving account (<code>savingBalance</code>)	SAV	Earns daily-accrued, monthly-distributed profit; held apart from the main wallet.
Share balance (<code>shareBalance</code>)	—	Business-share capital, accounted for as a liability; never auto-deducted for loans.
Loan account	LON	The member's borrowing; outstanding principal and repayments are tracked against it.

Account numbers are generated per member with type-specific prefixes (SAV / CUR / LON), so a number is self-describing at a glance.

REPORTING & RECONCILIATION

From rows to statements

Because every money movement is a ledger row, reports are derived from a single consistent source rather than reconstructed from disparate tools. P&L windows summarize income and expense over a period, statements present a member's or account's history, and per-branch analytics roll the same rows up by branch.

Branch attribution must cascade

Per-branch analytics read the `branchId` stamped on each row. When a member is reassigned or a branch is changed, that `branchId` must cascade to every dependent collection — Member, Customer, Loan, FinancialTransaction, Investment — or the branch's analytics silently read 0. The cascade is part of the update path, not an afterthought.

P&L & statements

Income/expense windows and account statements are computed directly from ledger rows, so they always tie back to recorded movements.

backfillLedger maintenance

A one-off `backfillLedger` maintenance script reconstructs missing historical ledger rows, keeping the audit trail complete for legacy data.

In one sentence

FinFlo posts every rupee to an auditable ledger with a single credit/debit convention, treats member shares as a liability, accrues and distributes profit on schedule, and wraps every multi-document money move in an idempotent transaction — so the books always reconcile.